C++ Assignment 3 – CO3

Student: Madhavan R
Register number: 192421174
Department: B.Tech Information Technology
Compiler used for verification: GCC g++ 14.2.0, C++17, Linux x86_64

> This report follows the seven questions in the supplied Assignment 3 document. The source document asks for individual execution and viva verification. Review every implementation and run it on your own machine before submission. Timings are machine-dependent and must be replaced with your own measurements if your faculty expects measurements from your computer.

Build and execution procedure

For each task folder:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main
./main
```

For memory checking on GCC/Clang:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer main.cpp -o main_asa
./main_asan
```

All seven `main.cpp` files were compiled and executed using GCC 14.2.0. Standard output is saved in each task's `output.txt`; compiler and sanitizer diagnostics are stored separately. AddressSanitizer completed without reported errors for the corrected Q2 and Q3 programs. The intentionally shallow-copy Q2 demonstration reports a double-free.

Q1 – Constructors and Constructor Overloading

Design and implementation

`Book` stores `title`, `author`, and `copies`. It provides a default constructor, title-only constructor, title-and-copies constructor, and full constructor. The shorter constructors delegate to the full constructor so the validation rule exists in one place. Every constructor uses a member initializer list. The full constructor throws `std::invalid_argument` if `copies < 0`; an invalid object is therefore not successfully constructed.

Pseudocode

```
FULL_CONSTRUCTOR(title, author, copies):
    initialize all data members
```

```
    if copies < 0:
        throw invalid_argument

DEFAULT_CONSTRUCTOR:
    delegate to FULL_CONSTRUCTOR("Untitled", "Unknown", 0)

TITLE_CONSTRUCTOR(title):
    delegate to FULL_CONSTRUCTOR(title, "Unknown", 0)

TITLE_AND_COPIES_CONSTRUCTOR(title, copies):
    delegate to FULL_CONSTRUCTOR(title, "Unknown", copies)
```

Compiler and language explanation

- The compiler implicitly declares a default constructor only when no user-declared constructor prevents it. Once a constructor is declared by the programmer, the compiler does not automatically add a no-argument constructor.
- A member initializer list directly initializes members. It is required for reference and `const` members and avoids default-construct-then-assign for class members.
- Members initialize in their declaration order, regardless of the order written in the initializer list. The `reorder_demo.cpp` experiment intentionally lists them in the opposite order and GCC emits `-Wreorder` plus an uninitialized-use warning. The resulting value of the first member is not reliable because it reads a member before that member has been initialized.
- Mark a converting constructor `explicit` when implicit conversion would be surprising or unsafe. For example, without `explicit`, `Book b = std::string("Dune");` can invoke a title constructor; with `explicit`, copy-initialization is rejected while `Book b(std::string("Dune"));` remains valid.
- A user-defined constructor can be overloaded by parameter list. Constructor delegation avoids repeating defaults and validation.

Three object states

Object	Constructor used	Title	Author	Copies
`empty`	default	Untitled	Unknown	0
`donated`	title-only	The Alchemist	Unknown	0
`full`	full	The C++ Programming Language	Bjarne Stroustrup	3

Verified output

```
Untitled | Unknown | copies=0
The Alchemist | Unknown | copies=0
Clean Code | Unknown | copies=4
The C++ Programming Language | Bjarne Stroustrup | copies=3
Rejected: copies cannot be negative
```

`array_without_default.cpp` intentionally fails to compile because `Book shelf[2]` requires a default constructor, but the sample class only has a title constructor. The compiler diagnostic is saved as `array_compile_error.txt`.

Test cases

Test	Input / operation	Expected result	Observed
Default construction	`Book empty;`	Safe defaults	Pass
Donated book	`Book("The Alchemist")`	Title set, other defaults	Pass
Full record	title, author, 3	All fields set	Pass
Negative copies	copies = -2	Exception; no valid object	Pass
Array without default constructor	`Book shelf[2]` in isolated example	Compile-time error	Error captured as intended
Wrong initializer order	`-Wreorder` demo	Warning	Warning captured as intended

Alternative: one constructor with default arguments

A single constructor such as `Book(std::string title = "Untitled", std::string author = "Unknown", int copies = 0)` can replace the overload set. It has the same O(1) initialization time and O(1) object storage. It is shorter, but overloads express distinct creation modes more clearly. Both designs should centralize validation in one constructor body; delegation makes that especially explicit.

Complexity: construction O(1) time and O(1) object space (excluding dynamically sized string contents, whose storage depends on title/author length).

Q2 – Copy Constructors: Shallow and Deep Copies

Design and implementation

`Prescription` owns a dynamically allocated array of `std::string`. The copy constructor allocates a new array and copies each medicine. Copy assignment first creates a replacement array, copies into it, then releases the old array; it checks `this != &other`. The destructor uses `delete[]`.

Pseudocode

```
COPY_CONSTRUCTOR(other):
    n = other.n
    allocate a new array of n strings
    copy each string from other

COPY_ASSIGNMENT(other):
    if this is not other:
        allocate temporary array sized to other.n
        copy all strings into temporary array
        delete[] current array
        take ownership of temporary array
        update n
    return *this

DESTRUCTOR:
    delete[] medicines
```

Memory diagrams

Shallow copy (incorrect):

```
original.medicines ----+----> [Paracetamol, ORS]
                       |
copy.medicines --------+
```

Both objects point to the same allocation. Mutating either object affects the other; when both destructors run, both attempt to free the same array (double delete).

Deep copy (correct):

```
original.medicines --------> [Paracetamol, ORS]
copy.medicines ------------> [Paracetamol, ORS]  (separate allocation)
```

The Rule of Three says that a class managing a resource with a destructor generally needs a copy constructor and copy-assignment operator as well. They must agree on ownership semantics. The copy constructor is used when an

object is initialized from another object, when an object is passed by value, and when an object is returned by value (subject to copy elision/move rules).

Passing by value makes a copy; passing by `const Prescription&` does not copy the object. A compiler-generated copy is appropriate when members already have correct value semantics (for example `std::string` or `std::vector`). A raw owning pointer generally requires custom ownership logic or, preferably, a standard container.

Required complete-code result

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The output proves that changing `p` does not alter `q` or `r`. Self-assignment `r = r` is also exercised.

Sanitizer investigation

- Correct deep-copy program: compiled and run with AddressSanitizer; no sanitizer diagnostic was produced.
- `shallow_copy_demo.cpp`: intentionally uses the compiler-generated shallow copy while both objects delete the same pointer. AddressSanitizer reported `attempting double-free`.
- The exact full sanitizer trace is in `shallow_asan_stderr.txt`.

Copy timing experiment (10 copies per size)

Recorded on the build environment; these are illustrative local measurements, not portable performance guarantees:

Medicines per prescription	10 deep copies
10	2 s
1,000	103 s
100,000	11,838 s

The program prints the measurements when run. The copy constructor also prints a marker, so output includes copy-count evidence. The experiment includes allocation and string-copy costs.

Alternative: `std::vector<std::string>`

Using `std::vector<std::string> medicines;` allows the compiler-generated copy operations to perform independent value copies. The class can follow the Rule of Zero and omit custom destructor/copy operations. Deep copy time is O(n) and additional space is O(n) for n medicine entries. A shallow pointer copy is O(1) time and O(1) extra space but is incorrect for shared ownership and unsafe destruction.

Complexity: deep copy O(n) time and O(n) additional space; self-assignment check O(1) before any copy work.

Q3 – Dynamic Constructors and Destructors

Design and implementation

`Event` dynamically creates an array of row pointers, then allocates one character array per row. `'.'` means available and `'X'` means booked. `book` and `cancel` validate indices. The destructor releases every row with `delete[]`, then releases the row-pointer array.

Pseudocode

```
CONSTRUCTOR(rows, seats):
    validate dimensions
    allocate rows row-pointers
    for each row:
        allocate seats characters
        fill with '.'

BOOK(row, seat):
    validate indices
    if already X: return false
    set X; increase booked count; return true

DESTRUCTOR:
    delete[] every row
    delete[] row-pointer array
    print closing summary
```

Allocation layout

```
seats ---> [row pointer 0] ---> ['.', 'X', '.', ...]
           [row pointer 1] ---> ['.', '.', 'X', ...]
           [row pointer 2] ---> ['.', '.', '.', ...]
```

A stack/local object is destroyed automatically when its scope ends (reverse construction order). A block-local object is destroyed at the closing brace. A heap object created by `new` remains alive until `delete` is executed. Forgetting `delete` leaks the object and its allocations. `delete[]` must match `new[]`; using scalar `delete` on an array is undefined behavior. A destructor has no parameters and no return type, and it cannot be overloaded because there is only one destruction operation for a given object.

Verified output

```
Construct event 2x3
. . .
. . X
Destroy event; booked=1
Construct event 1x2
Destroy event; booked=1
Construct event 0x5
Destroy event; booked=0
```

The heap event is destroyed exactly at `delete heap`. The local zero-row event is valid and performs no row allocation. The destructor is RAII: resource release is tied to object lifetime, including normal scope exit and exception unwinding.

Memory measurement

For `char` seats and `char*` row pointers, payload plus row-pointer bytes (excluding allocator metadata/alignment):

Seat map	Seat bytes	Row pointers (8-byte pointer assumption)	Approx. total
10×20	200	80	280 bytes
100×200	20,000	800	20,800 bytes
$1{,}000 \times 2{,}000$	2,000,000	8,000	2,008,000 bytes

Prediction – Logger program (written before execution)

Predicted exact output:

```
+A +B -B copyAc use -Ac +P -P end -A
```

Verified output is stored in `prediction/logger_prediction.output.txt` and matches the prediction. `a` is created first; `b` is created and destroyed at block exit; `use(a)` copies `a` into its by-value parameter (`Ac`), prints `use`, then destroys that parameter; `p` creates a heap Logger and `delete p` destroys it; `a` is destroyed at the end of `main`.

Alternative allocation designs

A contiguous block uses one allocation for `rows * seats` characters and indexes an element as `r * seatsPerRow + s`; access remains O(1), and allocation count drops to one. It removes the `rows` row pointers, reducing overhead and improving locality. `vector<vector<char>>` manages allocations automatically but usually still has one vector allocation per row plus outer storage. The row-pointer design performs `rows + 1` allocations.

Complexity: allocation and initialization O(rows × seats); seat access O(1); cleanup O(rows × seats) because each row allocation is released, with O(rows) pointer storage plus O(rows × seats) seat storage.

Q4 – Overloading Arithmetic and Stream Operators

Design and implementation

`Money` stores a single integer count of paise, avoiding binary floating-point rounding in currency arithmetic. It overloads `+`, `-`, `+=`, `==`, `<`, `<<`, and `>>`. `+` returns a new value; `+=` modifies the current object and returns `Money&` to support chaining. Stream insertion is a non-member friend because the left operand is an `std::ostream`.

Pseudocode

```
ADD(a, b): return Money(a.paise + b.paise)
SUBTRACT(a, b): return Money(a.paise - b.paise)
ADD_ASSIGN(a, b): a.paise += b.paise; return a
EQUAL(a, b): compare stored paise
PRINT(stream, money): print sign, rupees, decimal point, two paise digits
```

The expression `a + b` is equivalent to `a.operator+(b)` for this member overload. A friend/non-member `operator+` can support symmetric expressions such as `100 + m` and `m + 100` when an appropriate conversion exists; a member operator requires the left operand to be a `Money` object. `<<` must be non-member so the left operand remains the stream.

Verified output

```
Rs 200.00
true
Rs 1.25
-Rs 3.00
bill=Rs 200.00
```

The paise carry test is `Rs 0.75 + Rs 0.50 = Rs 1.25`. Negative values are represented with a leading minus sign; an application could instead reject negative balances depending on its billing rules.

Engineering decision

Addition, subtraction, comparison, and multiplication/division by a scalar quantity/rate can make sense. Multiplying one monetary amount by another mon-

etary amount usually has no useful currency interpretation and should not be provided. Multiplying money by a quantity or percentage is a more meaningful API.

Alternative and complexity

Friend/non-member operators and named functions (`add`, `equals`) all perform O(1) arithmetic and use O(1) extra space. The friend form gives symmetric operand conversion; named functions are explicit but less idiomatic for arithmetic notation. Integer paise avoids many floating-point representation issues. A 1,000,000-addition benchmark was executed for integer paise and `double` on this GCC 14.2.0 environment. The loops call non-inlined addition helpers and consume the final values to reduce dead-code elimination. Results (microseconds):

Optimization	Integer paise	double
`-O0`	2,359	6,350
`-O2`	1,088	1,438

The timings are environment-specific; the integer result is exact (75,000,000 paise), while repeated floating-point addition may accumulate representation error.

Q5 – Increment, Comparison and Subscript Operators

Design and implementation

`Time` stores elapsed seconds. Prefix `++` increments and returns a reference to the updated object. Postfix `++` is distinguished by a dummy `int` parameter, saves the old value, increments the object, and returns the old value by copy. `Playlist` has checked const and non-const `operator[]` overloads. Invalid indexes throw `std::out_of_range` rather than accessing invalid memory.

Pseudocode

```
PREFIX_INCREMENT:
    increment seconds
    return reference to this

POSTFIX_INCREMENT(dummy):
    save old Time
```

```
    increment seconds
    return saved old Time

PLAYLIST_INDEX(i):
    if i < 0 or i >= size: throw out_of_range
    return track at i
```

The const subscript overload returns `const std::string&`, preventing modification through a const playlist. The non-const overload returns `std::string&` and allows modification. Postfix generally needs an extra copy to preserve the old value, so prefix is preferred when the old value is not needed.

Verified main-program output

```
10 sec 12 sec 12 sec
rollover=3600 sec
Track B
Range error: playlist index
Track C
```

The example stores elapsed seconds; 3,599 + 1 = 3,600 seconds, which represents 1:00:00. A formatted clock display could split seconds into hours/minutes/seconds.

Prediction – Counter program (written before execution)

Predicted exact output:

```
5 7 7
```

Verified output is stored in `prediction/counter_prediction.output.txt` and matches. Starting from 5, postfix returns a copy containing 5 and then increments `c` to 6. Prefix increments `c` to 7 and returns the updated object, so `b` is 7 and `c` remains 7.

Test cases

Test	Expected	Result
`a = t++` from 10	`a=10`, `t=11`	Pass
`b = ++t` after above	`b=12`, `t=12`	Pass
playlist index 1	second track	Pass
playlist[-1]	range exception	Pass
const playlist index 2	read-only third track	Pass
59:59 + one second	1:00:00	Pass (seconds representation: 3600)

Alternative access and complexity

`at(i)` throws `std::out_of_range` on invalid indexes and is checked. Built-in-style unchecked indexing is faster by a small constant in some cases but can cause undefined behavior. All are O(1) time; checked indexing adds a comparison/branch and exception path. A 10,000,000-increment benchmark was run with non-inlined class operators and a consumed final counter value. Results (microseconds):

Optimization	Prefix	Postfix
`-O0`	23,366	19,331
`-O2`	20,102	22,003

The measurement is noisy and includes function-call overhead; it does not prove postfix is generally faster. The semantic distinction remains: postfix must preserve and return the old value, while prefix returns the updated object. For playlist indexing, the implementation has one range comparison before access; a separate 10-million-index microbenchmark is not included.

Q6 – Rules of Operator Overloading

Design and implementation

`Text` owns a dynamically allocated null-terminated character array. It implements constructor, deep-copy constructor, copy assignment with self-assignment protection, destructor, `+=`, `+`, `==`, `!=`, and stream insertion. `operator=` must be a member function in C++. `<<` is a friend/non-member because its left operand is an output stream.

Pseudocode

```
CONSTRUCTOR(s): allocate length(s)+1; copy s
COPY_CONSTRUCTOR(other): allocate separate buffer; copy characters
ASSIGN(other): if not self, allocate replacement, copy, free old, adopt replacement
DESTRUCTOR: delete[] buffer
PLUS(a,b): copy a, append b, return result
EQUALITY(a,b): compare character sequences
```

The operators `::`, `.`, `.*`, `?:`, and `sizeof` cannot be overloaded. Overloading cannot change an operator's precedence, associativity, or arity, and at least one operand must be a user-defined type. A friend is useful when a non-member operator needs private representation access, especially stream insertion; it should not be used unnecessarily.

Verified output

```
Hello Cloud Computing
copies=2
empty='' equal=true
This is a longer SMS message used to test dynamic allocation safely.
```

For `a + b + c`, the implementation copies the left operand to form each intermediate concatenation. With C++17 return-value elision, this run counted two explicit copy-constructor calls. The exact count can vary if the implementation or expression changes; the copy counter is included to make the behavior observable. Self-assignment of the empty string is tested.

Complexity

For lengths n1, n2, n3, left-associative `(a+b)+c` copies characters of `a` into the first temporary, appends `b`, then copies the first intermediate into the next temporary and appends `c`. Character work is O(n1+n2+n3) for these two concatenations, with temporary storage proportional to the intermediate result; peak extra storage is O(n1+n2+n3). Repeated `+` in a loop can become quadratic if it repeatedly copies the accumulated prefix. `+=` on a mutable buffer can reduce avoidable copies, and `std::string` should normally be preferred because it already manages allocation, copying, and exceptions safely.

Experimental comparison

The task source prints the custom class output and copy count. A 100,000-character append experiment was executed at both optimization levels, consuming the final string length:

Optimization	Custom `Text` append (s)	`std::string` append (s)
`-O0`	205,501	308
`-O2`	203,746	312

The custom benchmark reallocates and copies the complete character array on each append, producing $O(n^2)$ total character movement. `std::string` grows capacity geometrically and provides amortized O(1) append, so the measured gap is expected. The benchmark is illustrative and machine-dependent.

Q7 – Complex Number Class for AC Circuit Impedance

Stage 1 – Requirements

Store real and imaginary components as `double`. Provide default, real-only, and real-plus-imaginary constructors. Required operations: addition, subtraction, multiplication, division, equality, rectangular output, and magnitude. Division by zero impedance must throw an exception. Measurable checks: operator results match hand calculations within a tolerance; output includes real and imaginary components; zero denominator is handled without crashing.

Stage 2 – Design and formulas

For `z1 = a + jb`, `z2 = c + jd`:

- Addition: `(a+c) + j(b+d)`
- Subtraction: `(a-c) + j(b-d)`
- Multiplication: `(ac-bd) + j(ad+bc)`
- Division: `[(ac+bd) + j(bc-ad)] / (c²+d²)`
- Magnitude: `sqrt(a²+b²)` (implemented with `std::hypot`)
- Equality: compare components within a small tolerance because floating-point values are approximate.

Arithmetic operators are members because the left operand is a `Complex`; stream insertion is a friend/non-member because the left operand is `std::ostream`.

Stage 3 – Implementation and verification

For a series RLC impedance, use `Z = R + j L - j/( C)` with R=10 Ω, f=50 Hz, L=0.1 H, and C=100 F. Here =2 f 314.159 rad/s, XL 31.416 Ω, and XC 31.831 Ω, so the series impedance is approximately `10 - j0.415 Ω`, with magnitude approximately 10.009 Ω.

The second calculation combines `Z1 = R + jXL` in parallel with the capacitor impedance using `(Z1 × Z2)/(Z1 + Z2)`. The program prints the computed rectangular value and magnitude. Dividing by a zero `Complex()` throws `std::domain_error`, which is caught and reported.

Verified output

```
Series RLC impedance: 10.000 - j0.415 |Z|=10.009
Parallel impedance: 101.147 - j27.633 |Z|=104.854
Safe error: division by zero impedance
```

Stage 4 – Design defense

`std::complex<double>` already implements complex arithmetic and is preferable in production because it is standard, tested, and less error-prone. A custom class is useful for learning, specialized validation, domain-specific formatting, or instrumentation. One limitation is that exact equality is not generally appropriate for floating-point calculations; this implementation uses a fixed tolerance and does not provide configurable numerical tolerances or uncertainty handling.

Alternative: `std::complex<double>`

Replace the custom type with `std::complex<double>` and use `std::real`, `std::imag`, `std::abs`, and standard arithmetic. Each arithmetic operation remains O(1) time and O(1) space; storage is two doubles in both designs. The standard type reduces implementation work. Application-level zero-denominator validation should still be explicit if the program requires a controlled error rather than relying on floating-point exceptional values.

Final checklist before submission

- ☐ Create the repository using the exact required name `DSA01-<RegisterNumber>-Assignment3`.
- ☐ Make at least one separate commit per task; do not rely on a single final upload because the assignment checks commit history.
- ☐ Run each program on your own system and save your own terminal screenshots if required by the faculty.
- ☐ Replace illustrative benchmark measurements with your own measurements for Q4, Q5, and Q6 if requested.
- ☐ Read and understand every source file so you can explain and modify it during viva.